

Wi-Fi Jammer Implementation on Ath9k



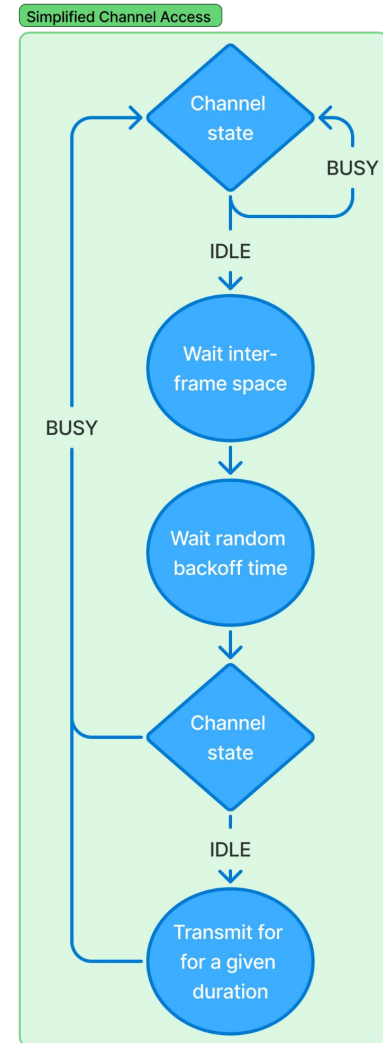
ECE 436 - Wireless Communications

Wi-Fi Fairness

- 802.11 protocol is designed to have distributed and **cooperative** channel access.
- Fairness depends on multiple **parameters** (AIFS, CWmin, CWmax)
- A modified driver can influence these parameters and gain **unfair advantages** over normal stations.

Normal 802.11 channel access

- **Sense** the channel.
- Wait until the channel is **idle**.
- Wait for an **inter-frame** space.
- Select a **random backoff** time and wait only if channel stays idle.
- If channel is still idle, transmit for a given **transmission opportunity** duration.



Inspiration: Advanced Wi-Fi Attacks Using Commodity Hardware

They used:

- open-source ath9k_htc firmware
- AR_DIAG_SW + AR_D_GBL_IFS_SIFS registers to disable carrier sense and alter the Short Interframe Space (SIFS)

They created 2 jammers:

- Continuous Jammer -> non stop jamming
- Selective Jammer -> listens for specific frames and then it jams them.



Objective

Our goal was to create an unfair station that stops other stations from transmitting.

- Research **EDCA** parameters and find out how they can influence throughput of other stations.
- Modify the ath9k so it can be **configured** to be either fair or unfair.
- Run controlled **experiments** with fair and unfair stations.
- Measure results and **analyze** throughput and **overall effect** on other fair stations.
- **Automate** the experiment flow.

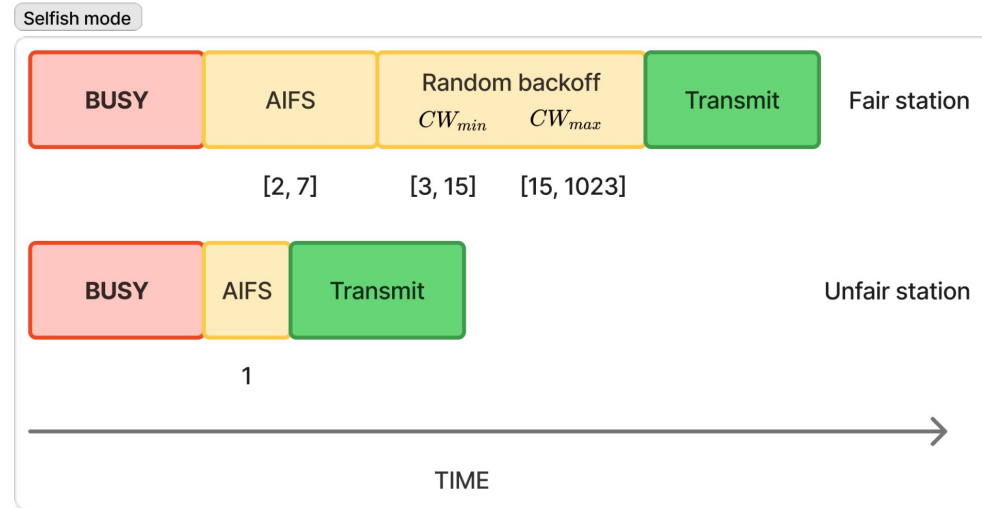
Modifications

<u>Parameter</u>	<u>Meaning</u>
<u>selfish_mode</u>	Aggressive EDCA queue parameters
<u>disable_backoff</u>	Disables random back-off waiting
<u>channel_idle</u>	Forces channel sense to idle
<u>selfish_txop_us</u>	Transmission opportunity override

Modification 1: CW & IFS (selfish_mode)

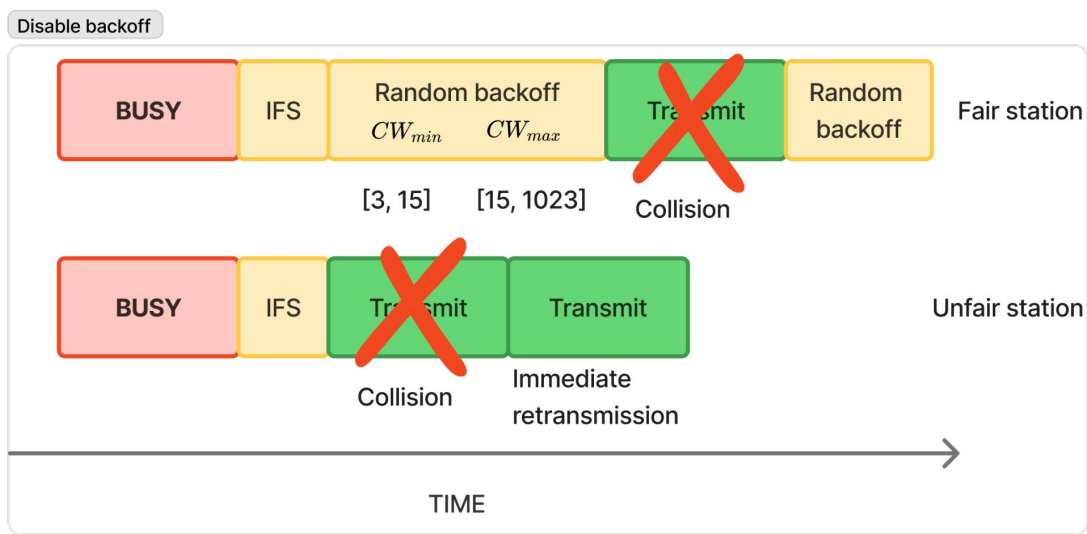
We make the station compete more aggressively by reducing waiting and random backoff before transmission.

- Normal stations wait for **AIFS** and then choose **random backoff**.
- We set AIFS equal to its **minimum (=1)** and the contention window minimum and maximum to **0**.
- This makes our station transmit **almost immediately** when channel becomes idle.



Modification 2: Disabling back-off (disable_backoff)

We set a bit on a hardware register (AR_D_GBL_IFS_MISC_IGNORE_BACKOFF) that **completely** bypasses the normal random backoff.

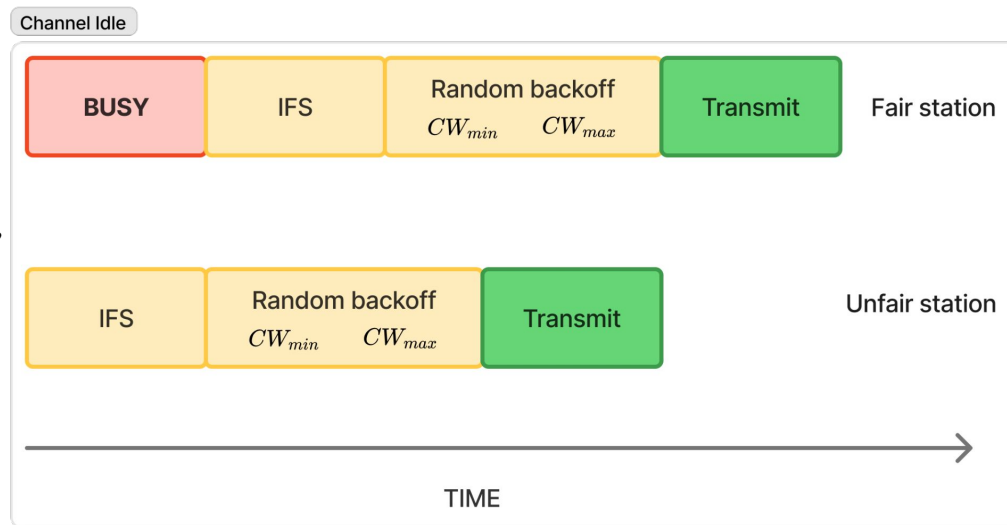


- Disables random backoff on **every occasion** even on collisions.
- **Lower level** modification than just changing CW_{min} and CW_{max} .
- Makes station even **more aggressive**.

Modification 3: Forced idle channel (chanel_idle)

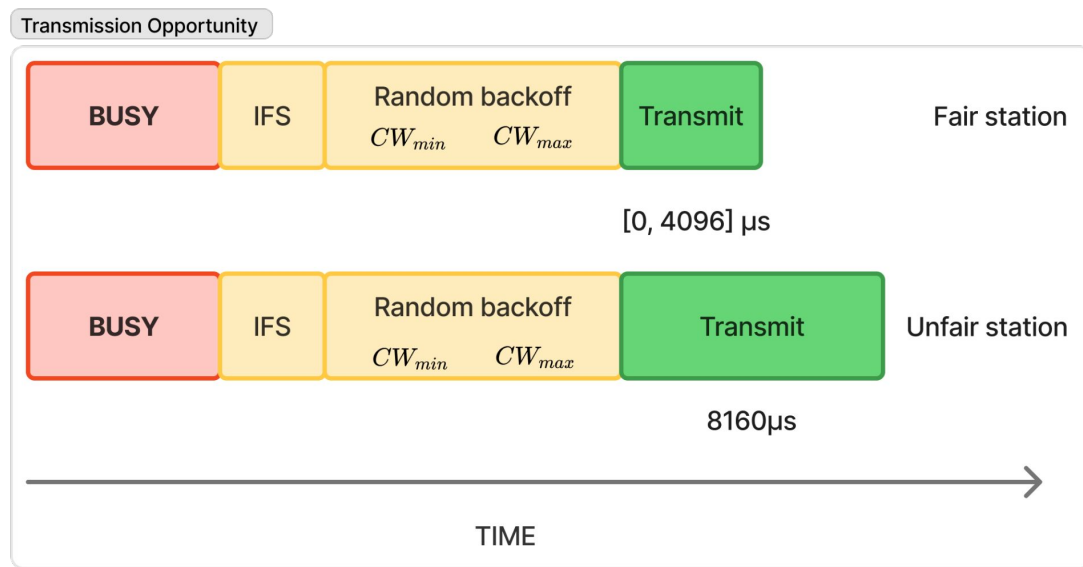
We set a bit on a register (AR_DIAG_FORCE_CH_IDLE_HIGH) and make the hardware behave as if the wireless channel is **always idle** even if there is activity on it.

- **Completely** bypasses the normal carrier-sense flow.
- Along with the other modification, it achieves the **most aggressive** behavior.
- Effectively **zero waiting** for channel to be released.



Modification 4: Transmission opportunity (selfish_txop_us)

We change how long the station may keep transmitting after it gains access to the wireless medium.



- Normal TXOP limits **prevent** a station from holding the medium for too long.
- Doesn't help to get the medium but **keeps** the medium for **more time** when it does.
- Makes station even **more aggressive**.

Implementation on backports driver

Declared parameters **selfish_mode**, **disable_backoff**, **chanel_idle** and **selfish_txop_us** to be able to experiment with the driver after compilation.

Functions used: **module_param** and **MODULE_PARM_DESC**

1. For selfish mode we overwrote the CWmin, CWmax and AIFS with their minimum values before they are committed to the registers in the ath9k_hw_resettxqueue function inside mac.c
2. For disable backoff version we just used REG_SET_BIT to activate the AR_D_GBL_IFS_MISC_IGNORE_BACKOFF register in the ath9k_hw_init_global_settings function inside hw.c and removed REG_CLR_BIT so that it stays turned on :)

Implementation on backports driver

3. For `chanel_idle` same as `disable_backoff` but with the register `AR_DIAG_FORCE_CH_IDLE_HIGH`
4. For `selfish_txop_us` we change the `tqi_burstTime` with out own `txop` parameter in the queue info struct and in order to avoid unwanted values we capped it at `AR_D_CHNTIME_DUR`

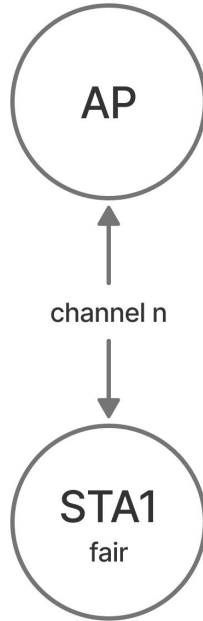
Automated flow

Created a shell script with an interactive menu and parameterized tests to:

- Be able to load on all nodes the correct image
- Upload/generate the patch of the driver
- Set up the network
- Execute the tests
- Send back the results and finally
- Generate the plots

Topology #1

Experiment 1

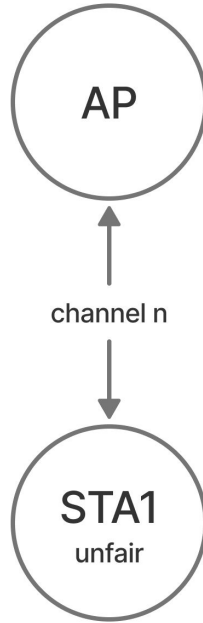


Baseline test:

- Fair node with unmodified driver and an AP
- Test the practical limits of a fair node
- Generated traffic: 150Mbps

Topology #2

Experiment 2

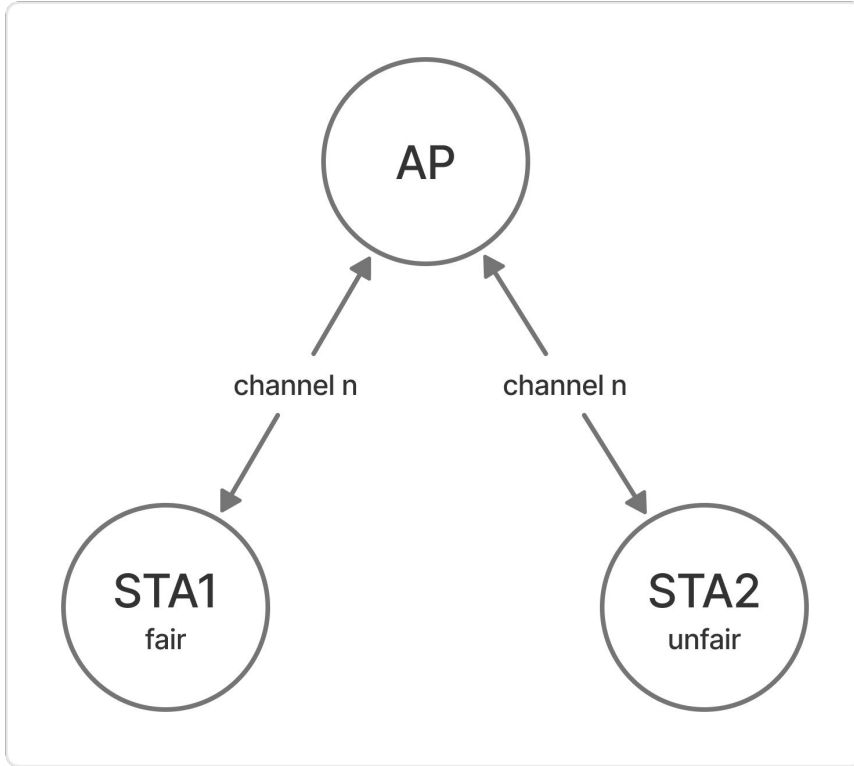


Baseline test:

- Unfair node with modified driver and an AP
- Test the practical limits of an unfair node
- Generated traffic: 150Mbps

Topology #3

Experiment 3

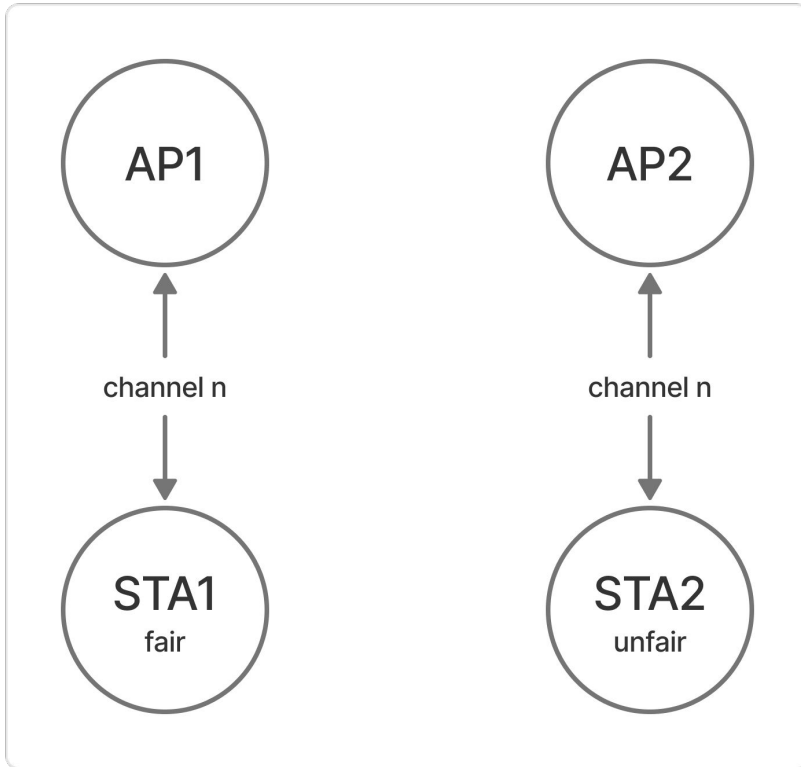


Jammer 1st test:

- One existing link of fair node + AP (with known IP)
- Introducing on the same network the jammer
- Both STAs try to generate 150Mbps

Topology #4

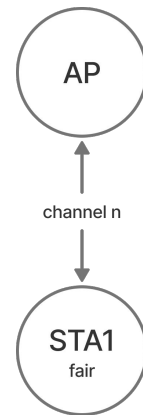
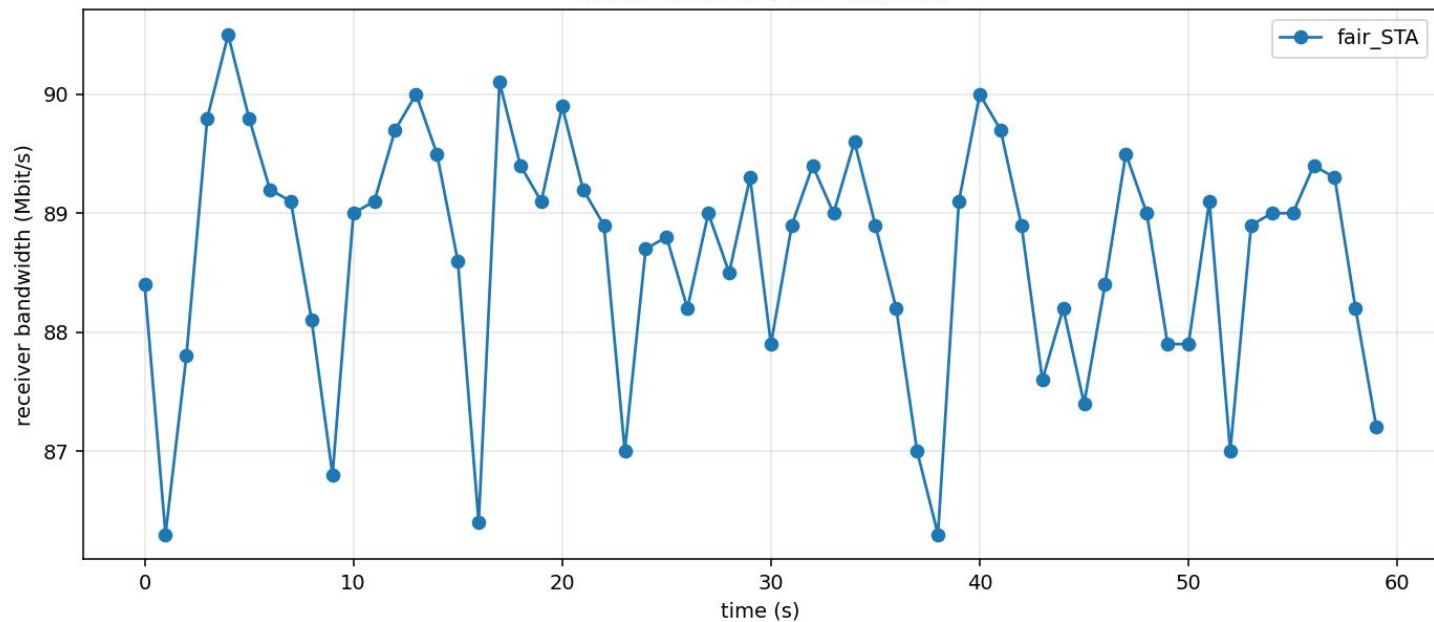
Experiment Type 4



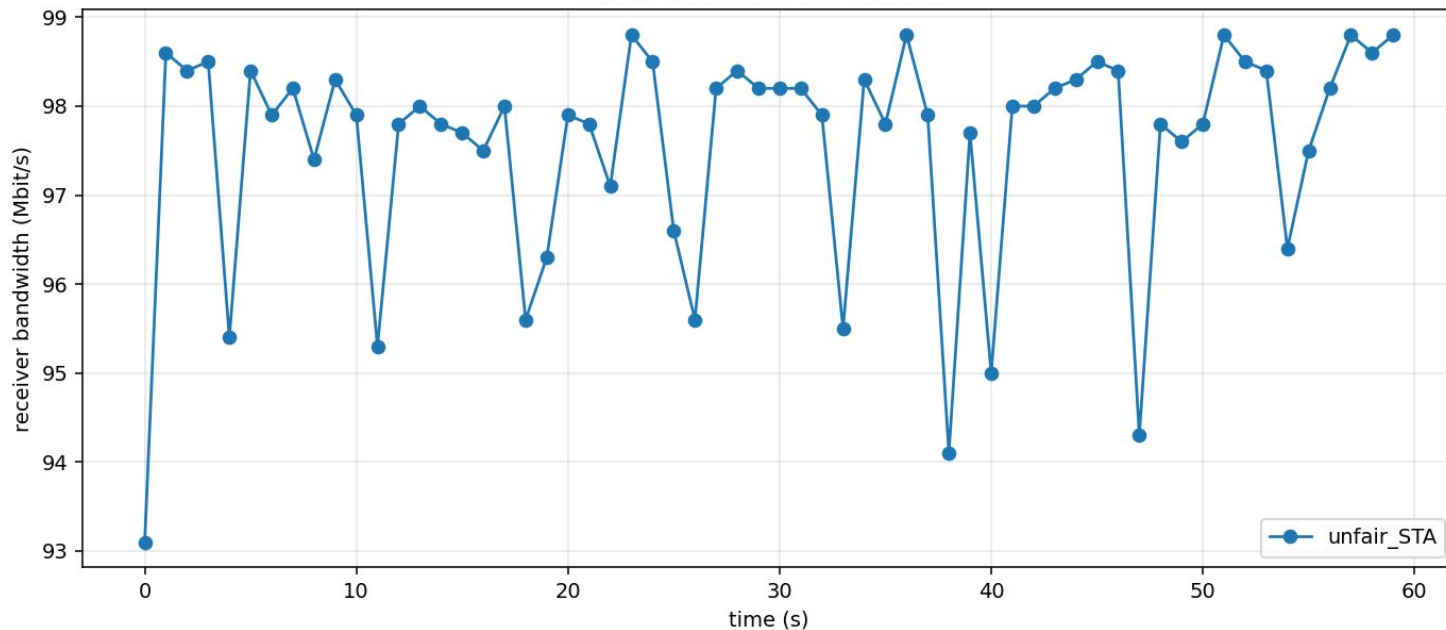
Jammer 2nd test:

- One existing link of fair node + AP (with unknown IP)
- Introducing on the same network the jammer with each one AP
- Both STAs try to generate 150Mbps
- Versions that the victim link is either on one channel or on multiple (via CSA)

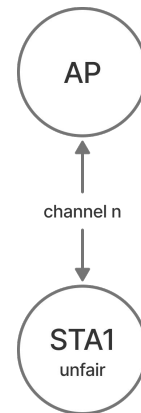
Experiment #1 Results



Experiment #2 Results

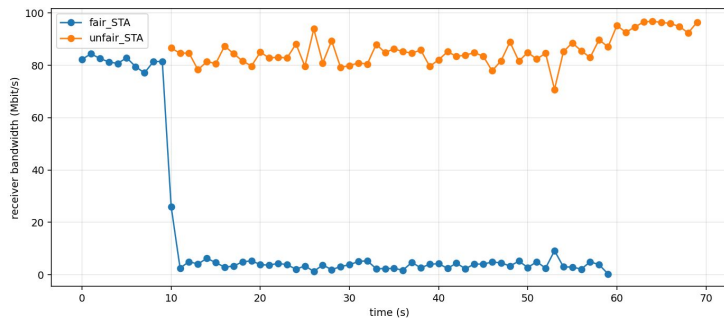


Unfair driver options: selfish_mode=1, disable_backoff=1, channel_idle=1, selfish_txop_us=1

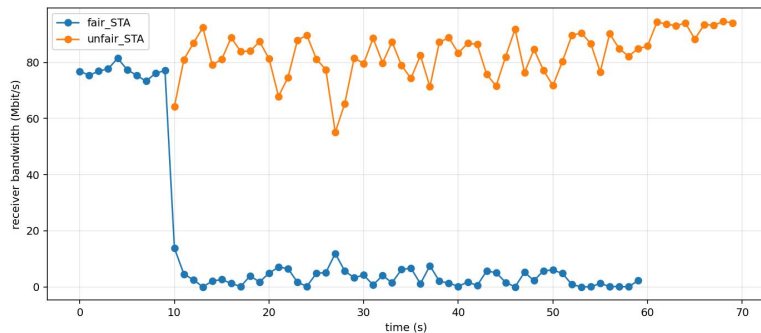


Experiment #3 Results

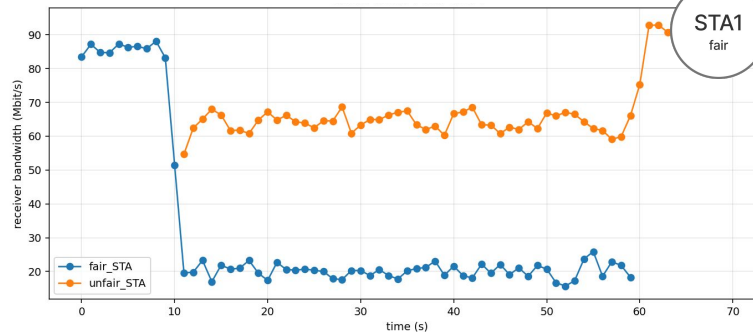
disable backoff



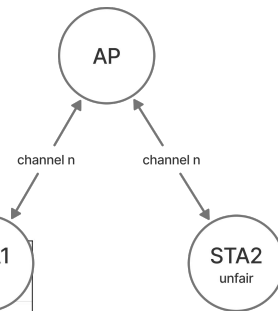
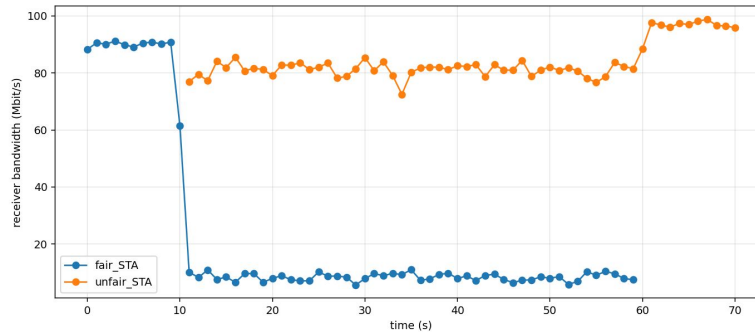
channel idle



selfish txop

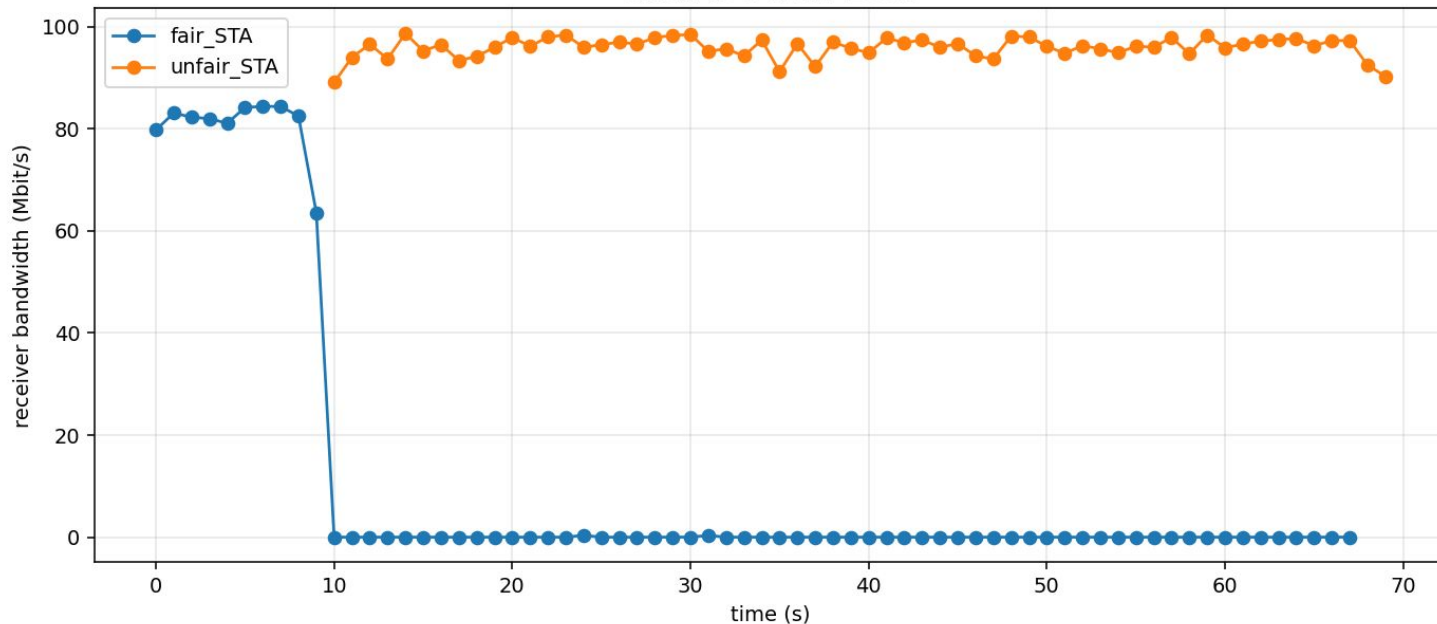


selfish mode

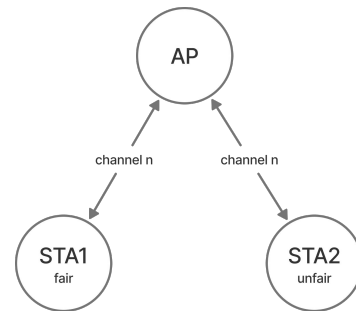


Experiment #3 Results

All options enabled

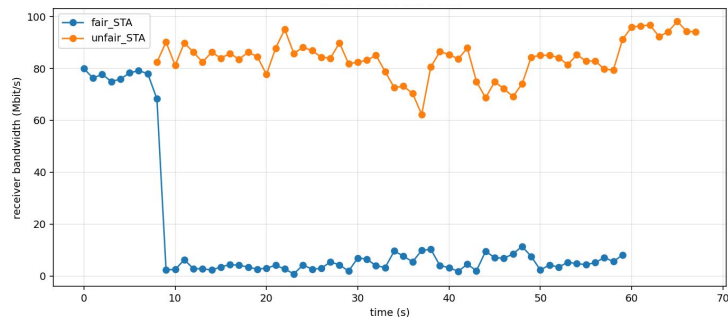


Unfair driver options: selfish_mode=1, disable_backoff=1, channel_idle=1, selfish_txop_us=8100

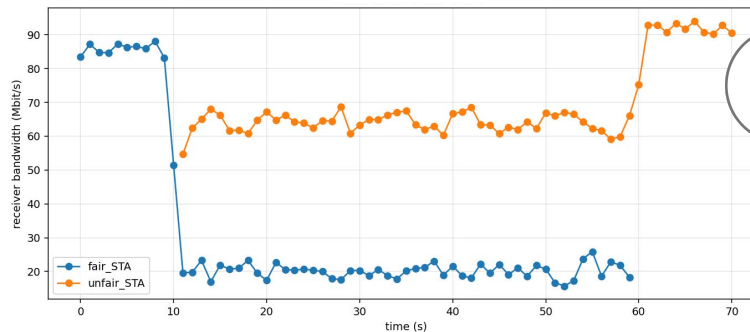


Experiment #4 Results

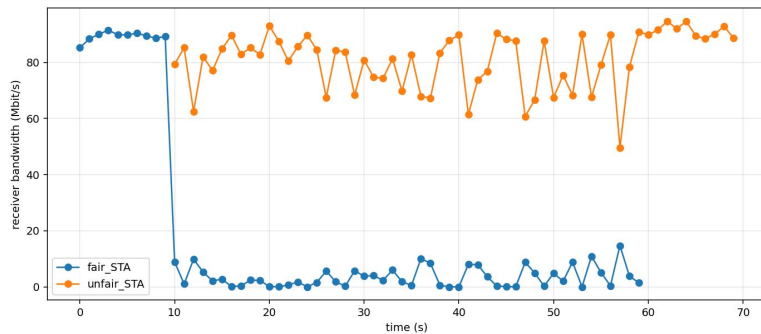
disable backoff



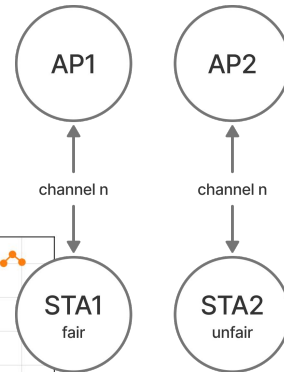
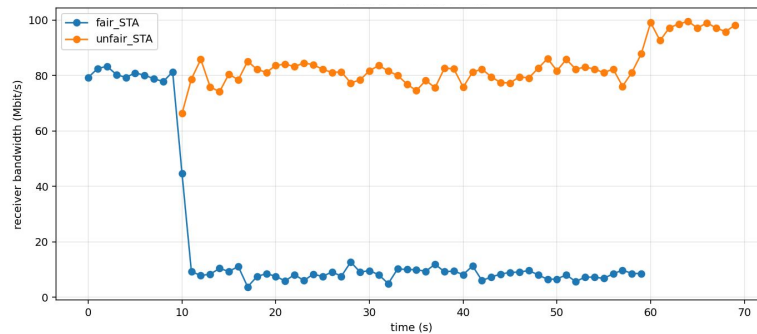
selfish txop



channel idle

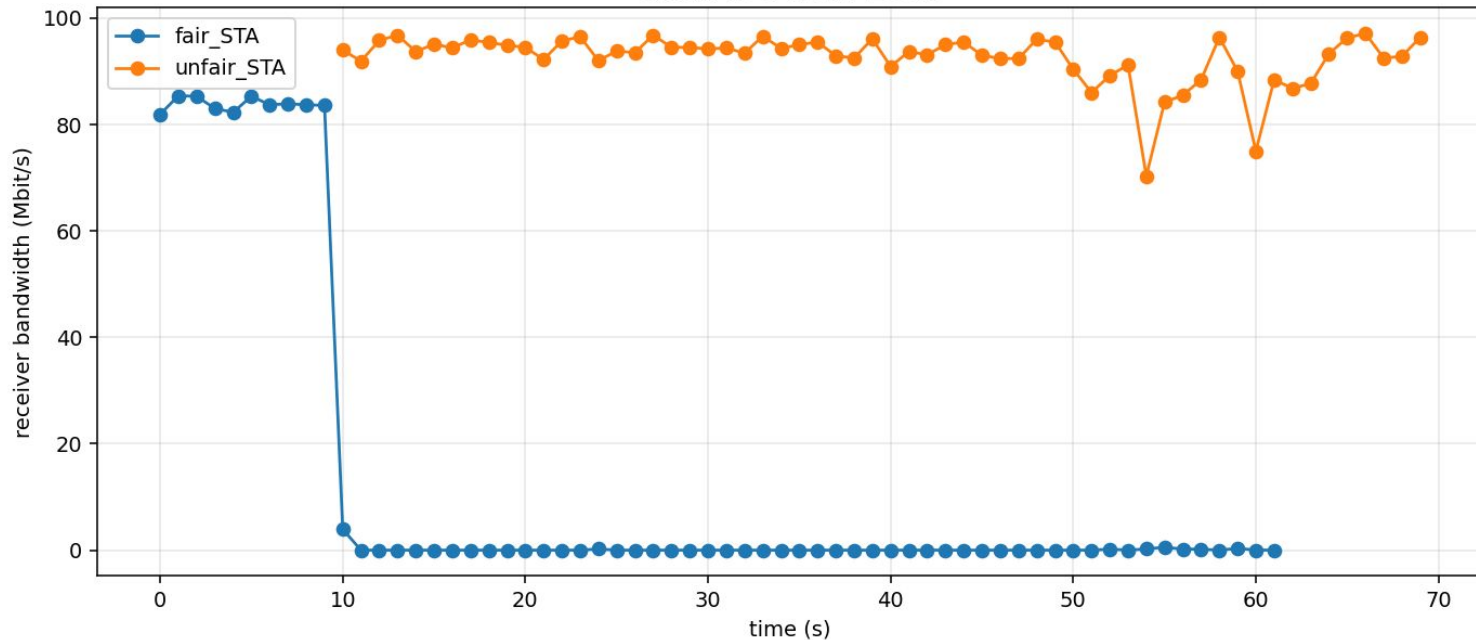


selfish mode

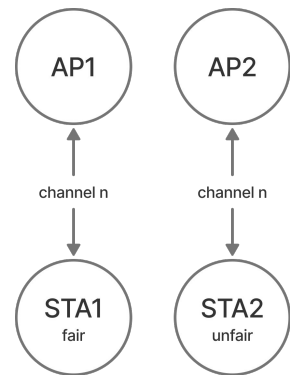


Experiment #4 Results

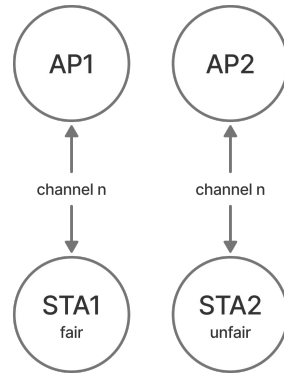
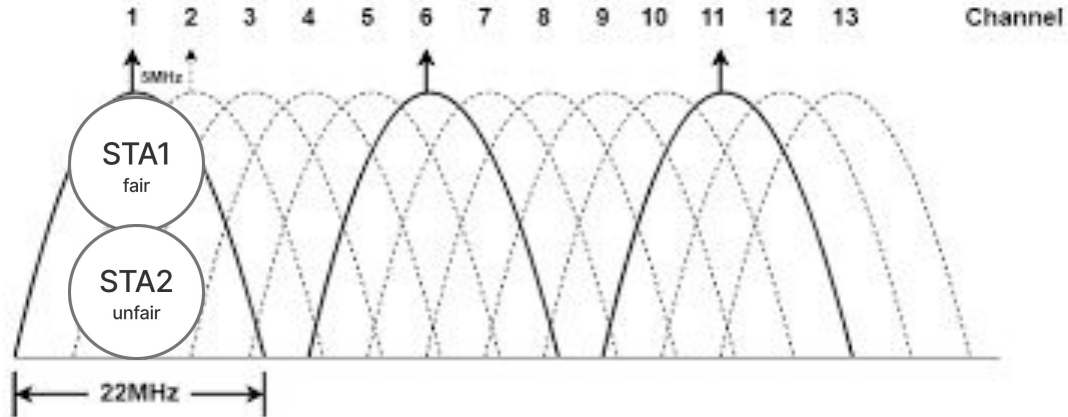
All options enabled



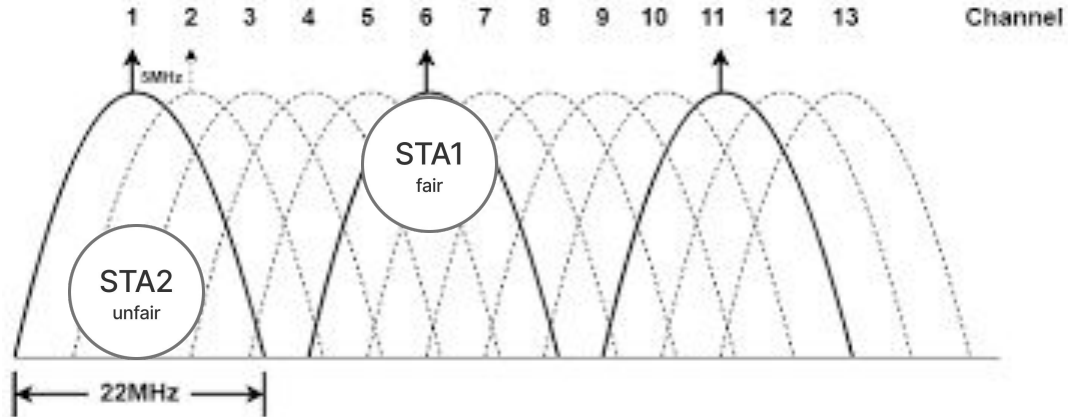
Unfair driver options: selfish_mode=1, disable_backoff=1, chanel_idle=1, selfish_txop_us=8100



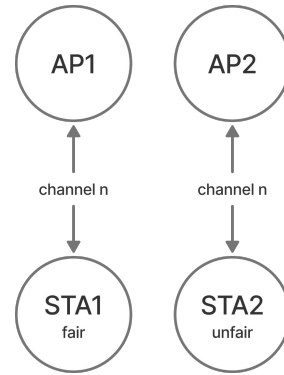
Experiment #5



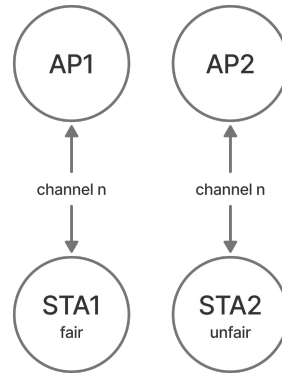
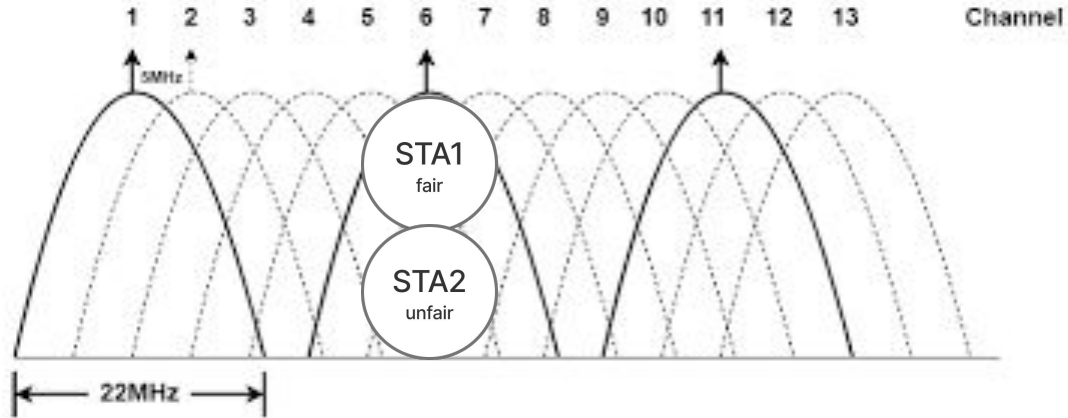
Experiment #5



Fair node hops to **channel 6**



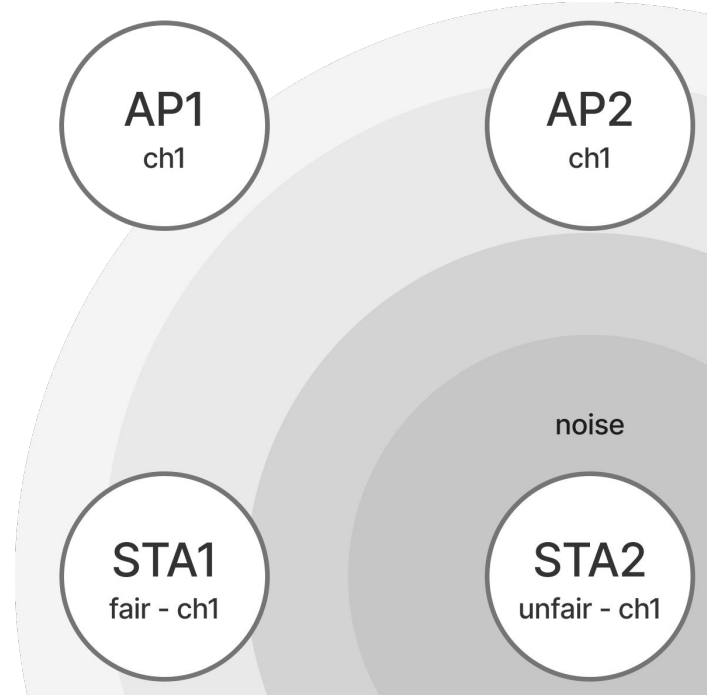
Experiment #5



Unfair node follows

Experiment #5

- Two links:
 - **victim:** AP1-STA1
 - **attacker:** STA2-AP2
- STA2 transmits **noise**

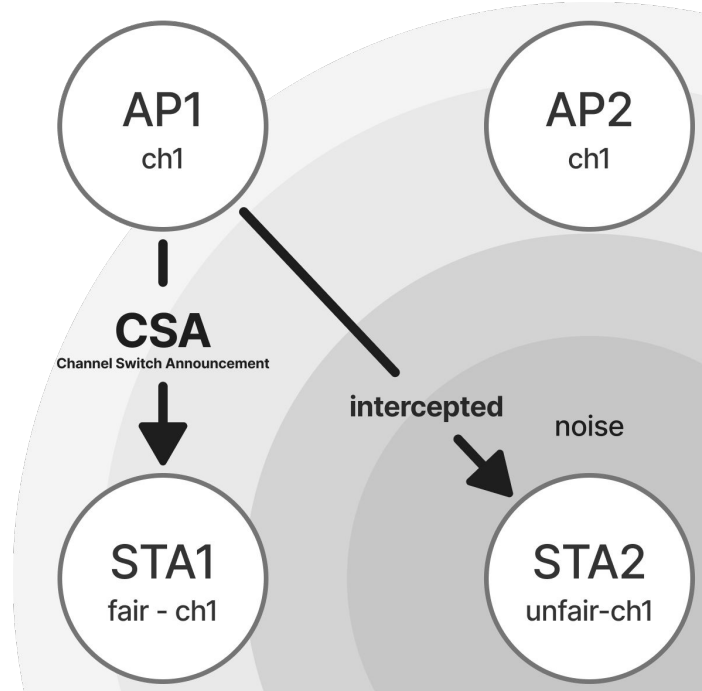


Experiment #5

- AP1 sends **Channel Switch Announcement (CSA)** to victim
- Attacker **intercepts** the beacon frames

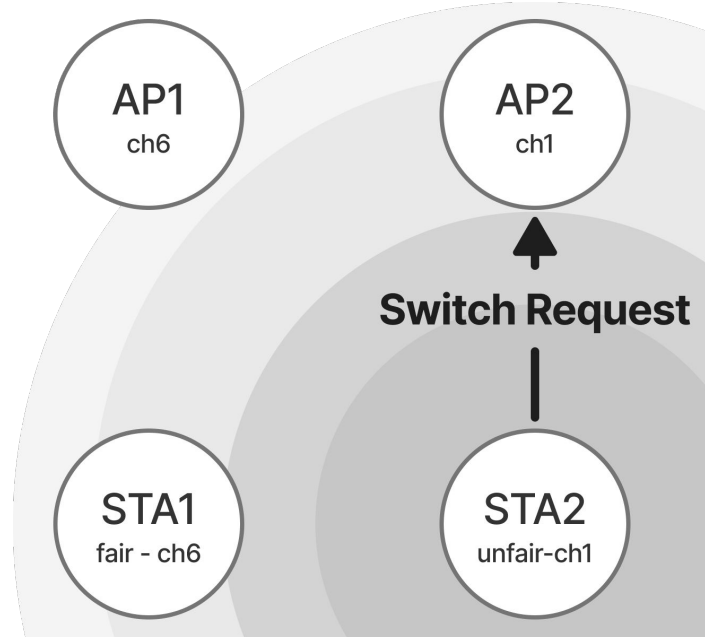
wlan.fc.type_subtype == 0x08 && wlan.bssid == 04:54:53:00:dd:b2 && wlan.tag.number == 37					
No.	Time	Source	Destination	Protocol	Length Info
76887	12.039686	Apple_00:dd:b2	Broadcast	802.11	229 Beacon frame,
77756	12.140621	Apple_00:dd:b2	Broadcast	802.11	229 Beacon frame,
78626	12.243868	Apple_00:dd:b2	Broadcast	802.11	229 Beacon frame,
79507	12.348491	Apple_00:dd:b2	Broadcast	802.11	229 Beacon frame,

Tag length: 4
DTIM count: 1
DTIM period: 2
Bitmap control: 0x00
Partial Virtual Bitmap: 00
Tag: Channel Switch Announcement Mode: 0, Number: 6, Count: 4
Tag Number: Channel Switch Announcement (37)



Experiment #5

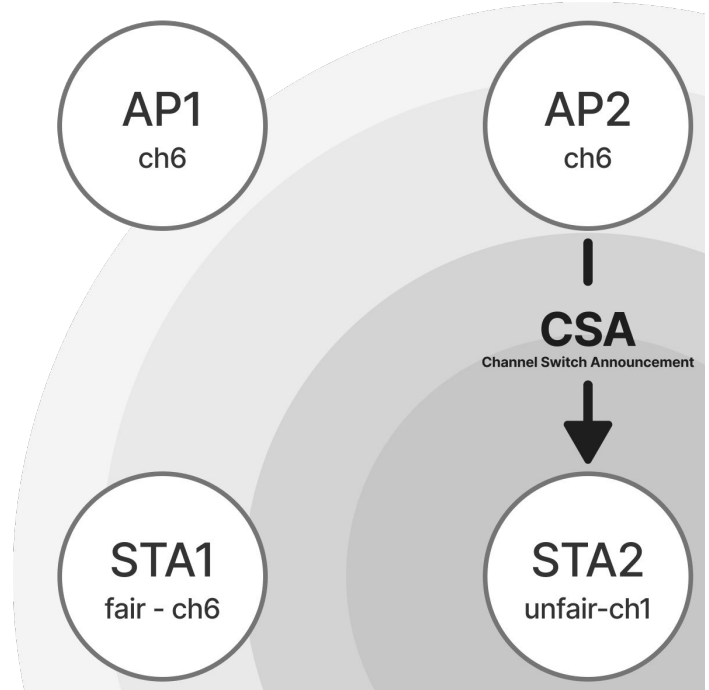
- Victim link **moved** to channel 6
- Attacker **requests** its AP to **switch channels**
- Attacker's AP runs **UDP server** listening to requests



Experiment #5

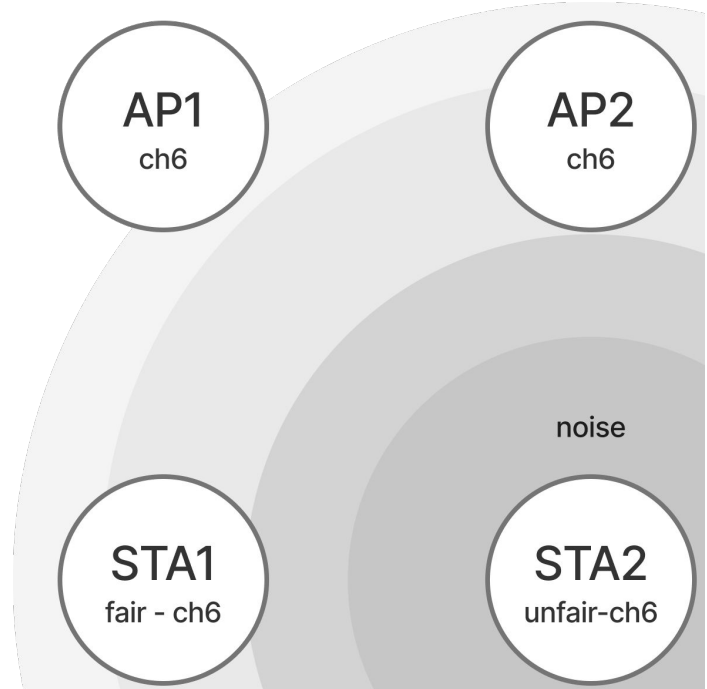
- AP2 sends **CSA** to attacker:

```
hostapd_cli -p /var/run/hostapd -i  
wlan0 chan_switch 5 2437
```

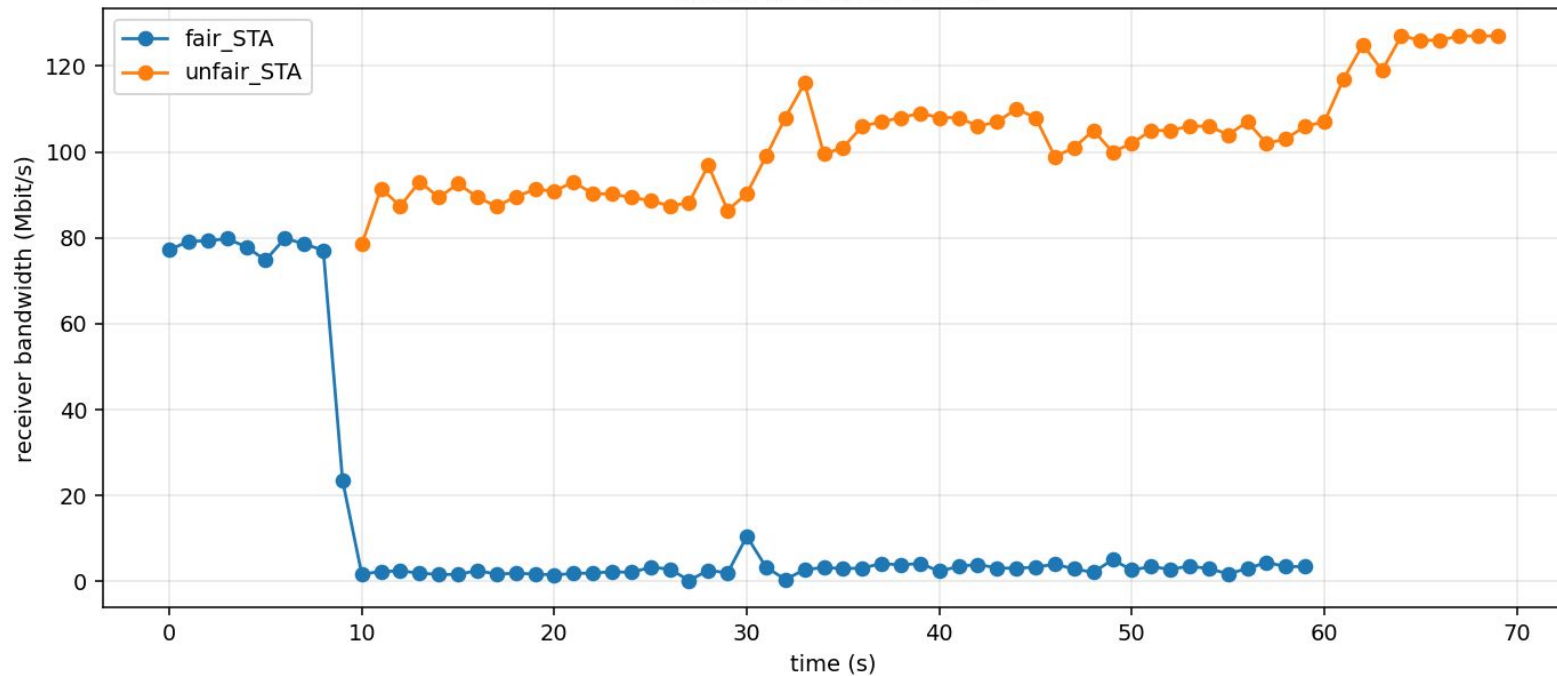


Experiment #5

- Continues **jamming** on **channel 6**



Experiment #5



Unfair driver options: selfish_mode=1, disable_backoff=1, selfish_txop_us=8100

Thank you!

